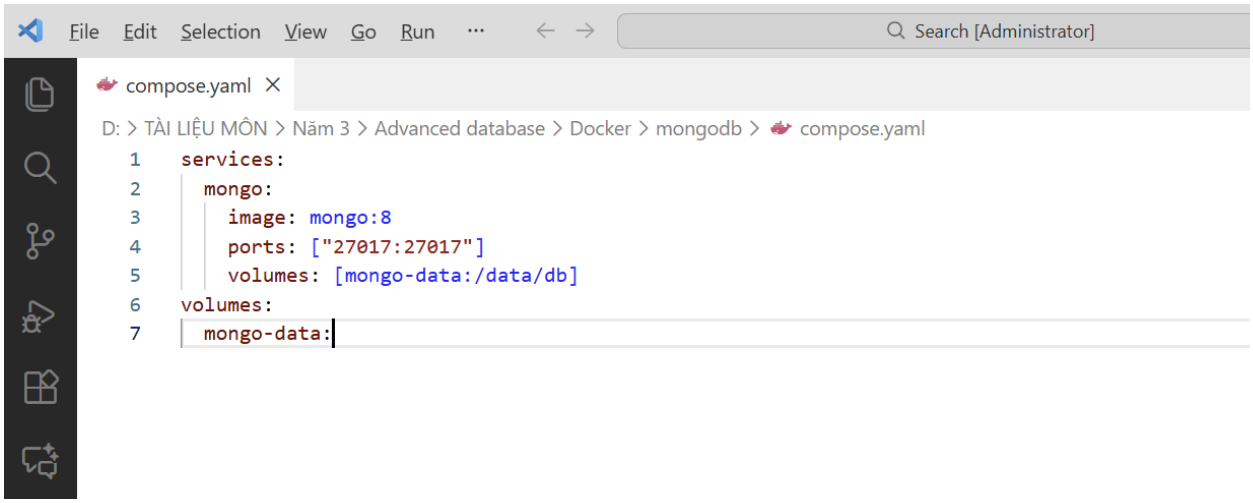
1. Chạy MongoDB:

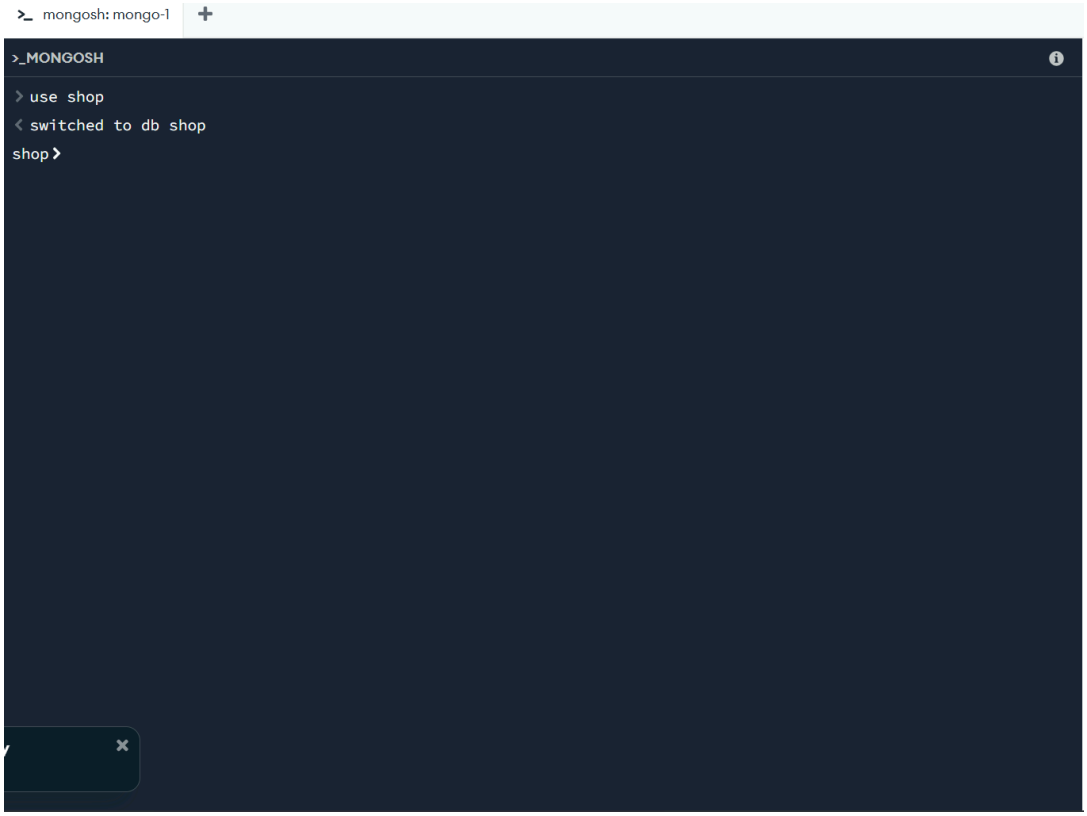
- file `compose.yaml`:



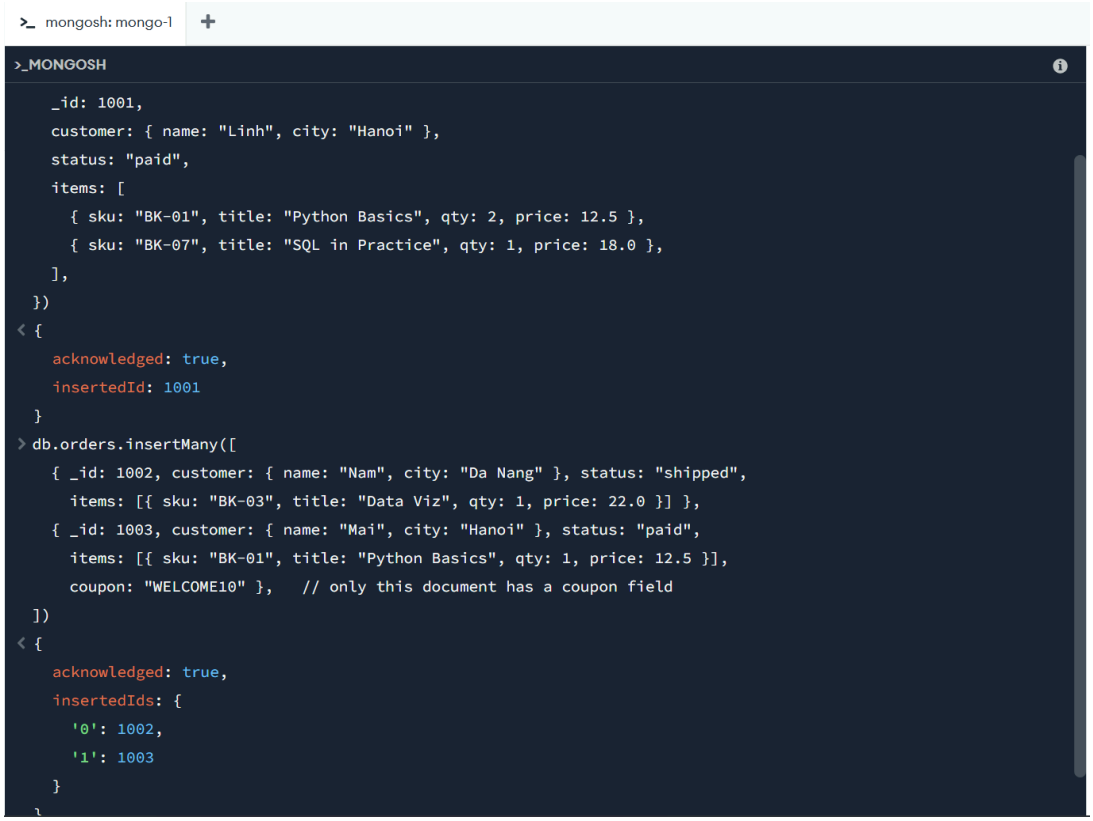
- khởi động lại container:

```
Admin@DESKTOP-TH2Q8EU MINGW64 /d/TÀI LIỆU MÔN/Năm 3/Advanced database/Docker/mongodb
$ docker compose up -d
[+] up 1/1
 ✓ Container mongodb-mongo-1 Running
```

- chạy “use shop” sau khi mở shell:

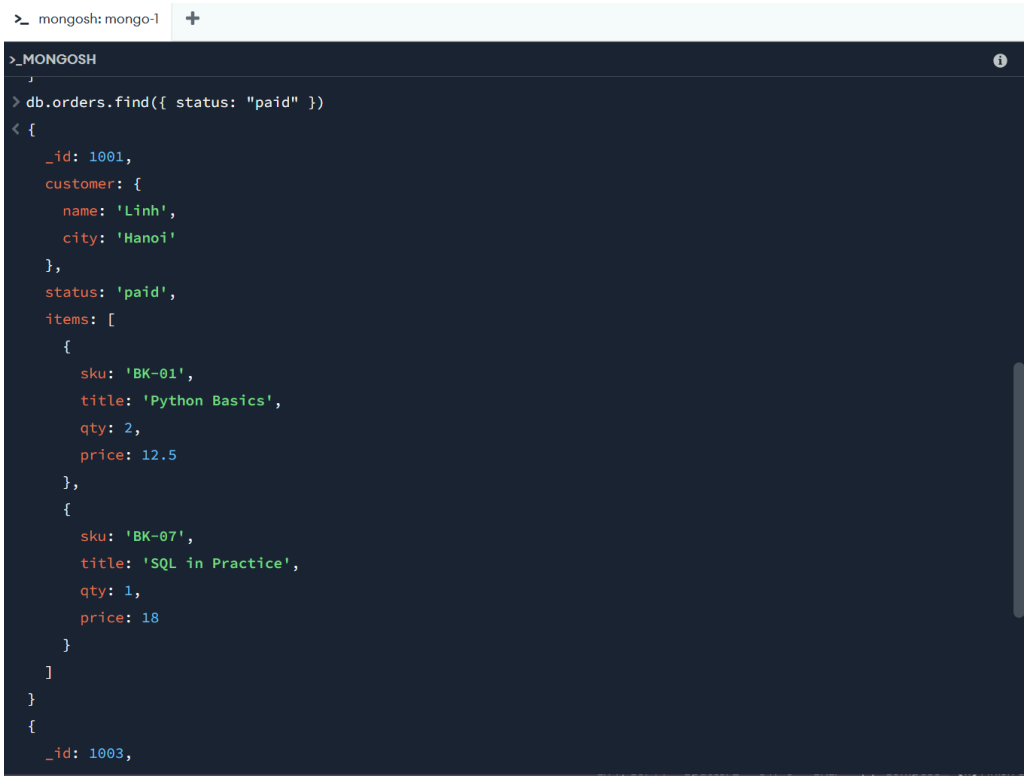


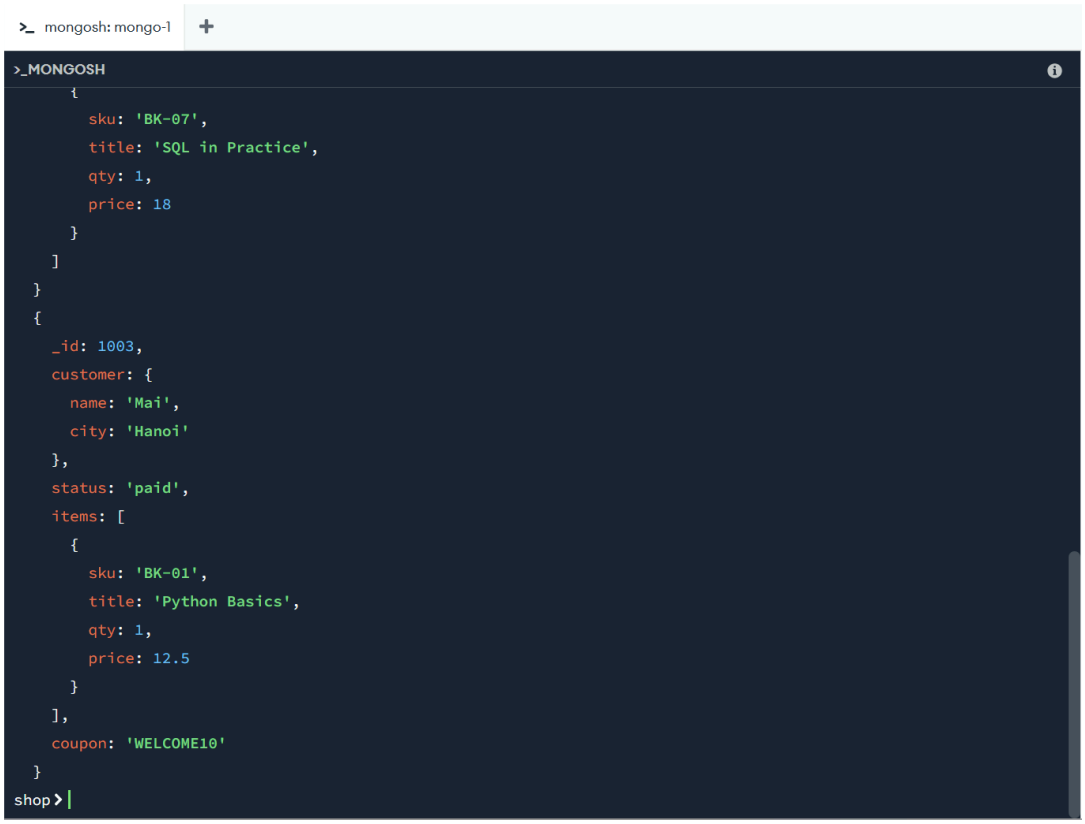
2. Create: Thêm document



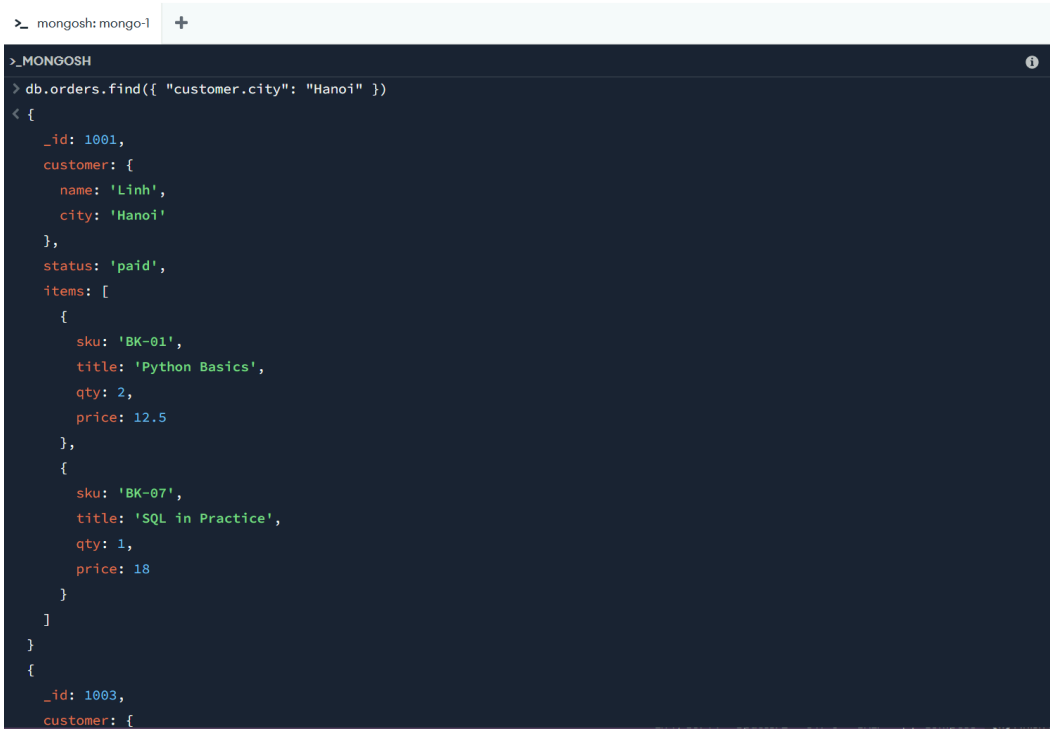
3. Read: truy vấn

- `db.orders.find({ status: "paid" })`



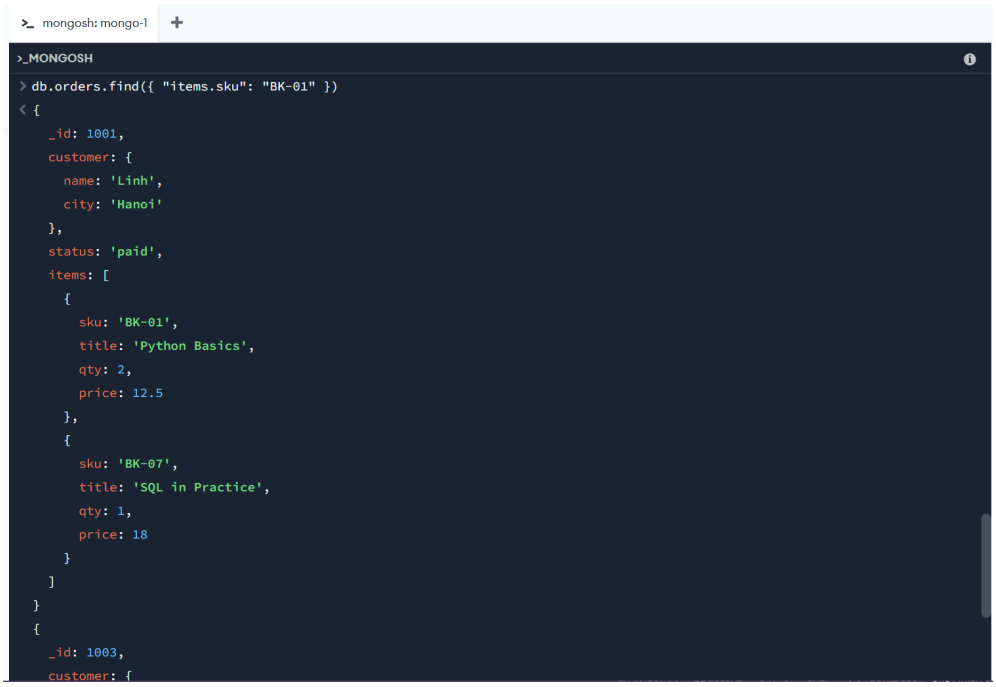


- `db.orders.find({ "customer.city": "Hanoi" })`



```
  _id: 1003,
  customer: {
    name: 'Mai',
    city: 'Hanoi'
  },
  status: 'paid',
  items: [
    {
      sku: 'BK-01',
      title: 'Python Basics',
      qty: 1,
      price: 12.5
    }
  ],
  coupon: 'WELCOME10'
}
shop >
```

- `db.orders.find({ "items.sku": "BK-01" })`




```
{
  _id: 1003,
  customer: {
    name: 'Mai',
    city: 'Hanoi'
  },
  status: 'paid',
  items: [
    {
      sku: 'BK-01',
      title: 'Python Basics',
      qty: 1,
      price: 12.5
    }
  ],
  coupon: 'WELCOME10'
}
shop >
```

- `db.orders.find({ status: "paid", coupon: { $exists: true } })`

```
> db.orders.find({ status: "paid", coupon: { $exists: true } })
< {
    _id: 1003,
    customer: {
      name: 'Mai',
      city: 'Hanoi'
    },
    status: 'paid',
    items: [
      {
        sku: 'BK-01',
        title: 'Python Basics',
        qty: 1,
        price: 12.5
      }
    ],
    coupon: 'WELCOME10'
  }
shop >
```

- `db.orders.find({ "items.price": { $gt: 20 } })`

```
> db.orders.find({ "items.price": { $gt: 20 } })
< {
    _id: 1002,
    customer: {
      name: 'Nam',
      city: 'Da Nang'
    },
    status: 'shipped',
    items: [
      {
        sku: 'BK-03',
        title: 'Data Viz',
        qty: 1,
        price: 22
      }
    ]
  }
shop >
```

- `db.orders.find({ status: { $in: ["paid", "shipped"] } },
  { _id: 1, status: 1 }) // projection: only these fields
  .sort({ _id: -1 })
  .limit(2)`

```
}
> db.orders.find({ status: { $in: ["paid", "shipped"] } },
                 { _id: 1, status: 1 })                    // projection: only these fields
    .sort({ _id: -1 })
    .limit(2)
< {
    _id: 1003,
    status: 'paid'
  }
  {
    _id: 1002,
    status: 'shipped'
  }
shop >
```

4. Update & delete

- db.orders.updateOne({ _id: 1001 }, { $set: { status: "shipped" } })`

```
> db.orders.updateOne({ _id: 1001 }, { $set: { status: "shipped" } })
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
shop >
```

- `db.orders.updateMany({ status: "paid" }, { $set: { reviewed: false } })`

```
> db.orders.updateMany({ status: "paid" }, { $set: { reviewed: false } })
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

- `db.orders.updateOne({ _id: 1002 }, { $push: { tags: "gift" } })`

```
> db.orders.updateOne({ _id: 1002 }, { $push: { tags: "gift" } })     // append to an array
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

- `db.orders.updateOne({ _id: 1003 }, { $unset: { coupon: "" } })`

```
> db.orders.updateOne({ _id: 1003 }, { $unset: { coupon: "" } })      // remove a field
< {
    acknowledged: true,
    insertedId: null,
    matchedCount: 1,
    modifiedCount: 1,
    upsertedCount: 0
  }
```

- `db.orders.deleteOne({ _id: 1003 })`

```
> db.orders.deleteOne({ _id: 1003 })
< {
    acknowledged: true,
    deletedCount: 1
  }
shop >
```

5. Aggregation pipeline

```
>_MONGOSH
> db.orders.aggregate([
    { $match: { status: { $in: ["paid", "shipped"] } } },        // WHERE
    { $unwind: "$items" },                                       // one document per item
    { $group: {                                                  // GROUP BY
        _id: "$items.sku",
        units: { $sum: "$items.qty" },
        revenue: { $sum: { $multiply: ["$items.qty", "$items.price"] } },
    } },
    { $sort: { revenue: -1 } },                                  // ORDER BY
    { $limit: 5 },
  ])
< {
    _id: 'BK-01',
    units: 2,
    revenue: 25
  }
  {
    _id: 'BK-03',
    units: 1,
    revenue: 22
  }
  {
    _id: 'BK-07',
    units: 1,
    revenue: 18
  }
```

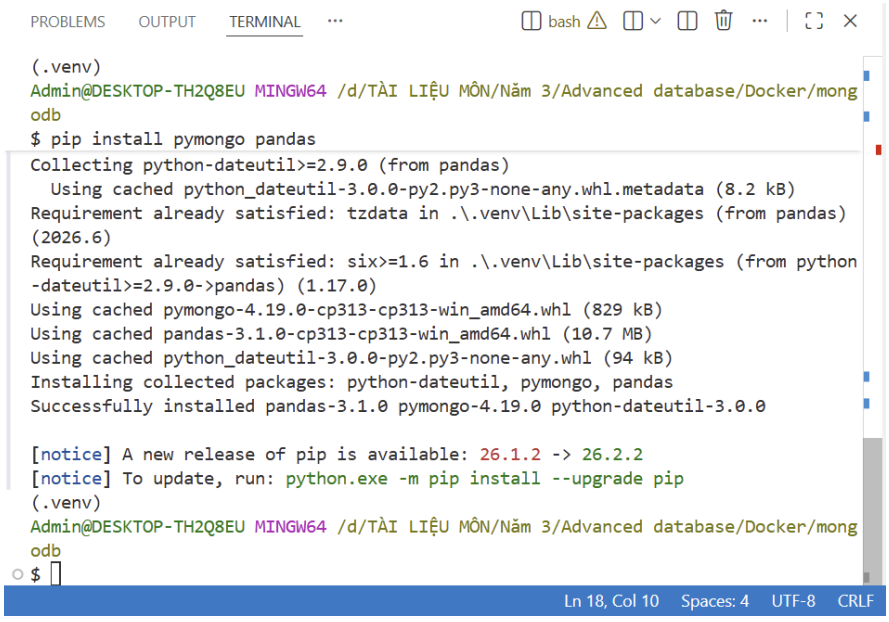
6. Index

```
>_MONGOSH
> db.orders.createIndex({ "customer.city": 1, status: 1 })
< customer.city_1_status_1
> db.orders.find({ "customer.city": "Hanoi", status: "paid" }).explain("executionStats")
< {
    explainVersion: '1',
    queryPlanner: {
      namespace: 'shop.orders',
      parsedQuery: {
        '$and': [
          {
            'customer.city': {
              '$eq': 'Hanoi'
            }
          },
          {
            status: {
              '$eq': 'paid'
            }
          }
        ]
      },
      indexFilterSet: false,
      queryHash: '534839F3',
      planCacheShapeHash: '534839F3',
      planCacheKey: '59FB1622',
      optimizationTimeMillis: 0,
```

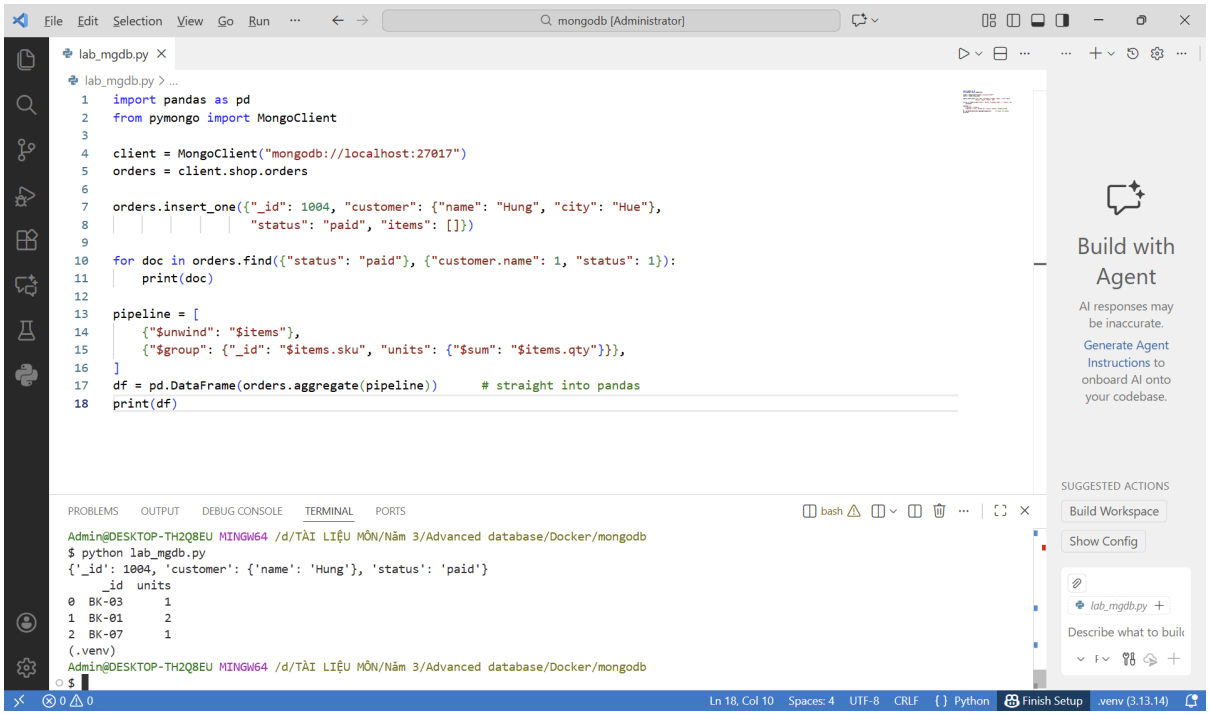
winningPlan gồm FETCH ->IXSCAN trên index customer.city_1_status_1, không có COLLSCAN

6. MongoDB từ Python

Cài thư viện

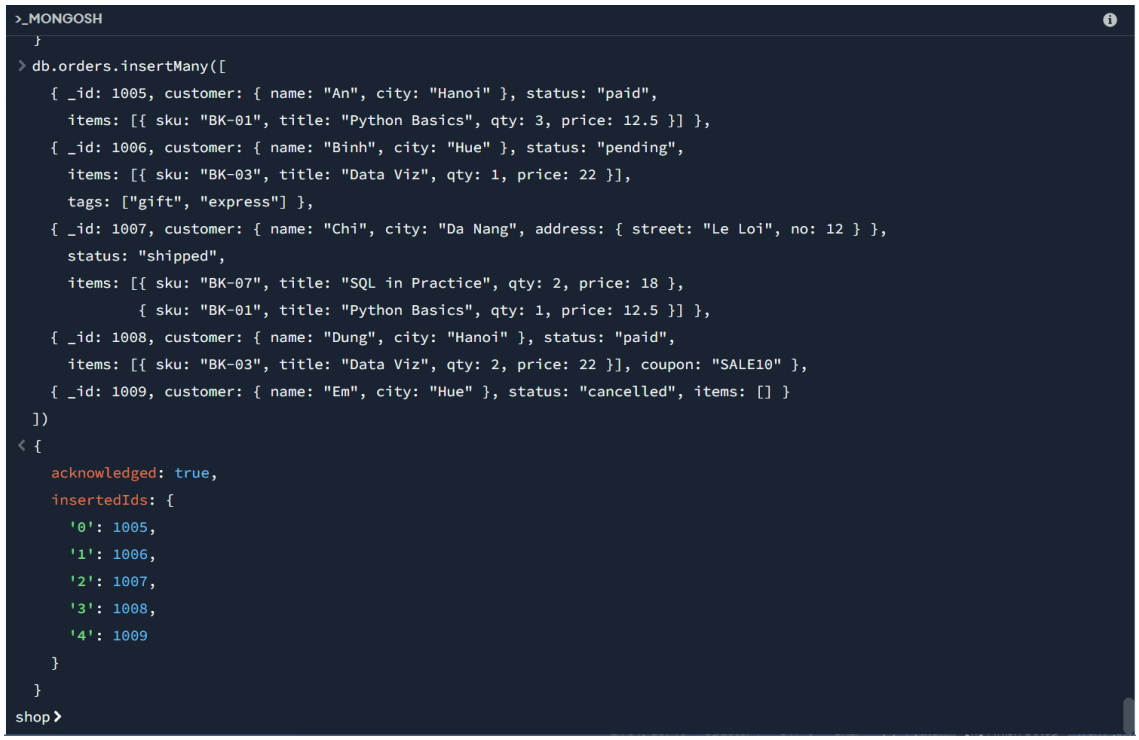


File python và output:



Bài tập

1.Thêm năm đơn hàng có hình dạng khác nhau, rồi tìm mọi đơn chứa một `sku` cho trước.



Kết quả đúng là các đơn 1001, 1005, 1007:

```
> db.orders.find({ "items.sku": "BK-01" }, { "customer.name": 1 })
< {
    _id: 1001,
    customer: {
      name: 'Linh'
    }
  }
  {
    _id: 1005,
    customer: {
      name: 'An'
    }
  }
  {
    _id: 1007,
    customer: {
      name: 'Chi'
    }
  }
shop >
```

2. Viết một aggregation trả về doanh thu theo thành phố.

```
> db.orders.aggregate([
    { $unwind: "$items" },
    { $group: {
        _id: "$customer.city",
        revenue: { $sum: { $multiply: ["$items.qty", "$items.price"] } }
    } },
    { $sort: { revenue: -1 } }
  ])
< {
    _id: 'Hanoi',
    revenue: 124.5
  }
  {
    _id: 'Da Nang',
    revenue: 70.5
  }
  {
    _id: 'Hue',
    revenue: 22
  }
shop >
```

Kết quả: Hanoi 124.5, Da Nang 70.5, Hue 22

3. Tạo một index và so sánh `explain()` trước và sau trên 10.000 document sinh ngẫu nhiên.

- Tạo 10000 document ngẫu nhiên:

```
> db.orders_big.drop()

  for (let i = 0; i < 10000; i++) {
    db.orders_big.insertOne({
      city: ["Hanoi", "Hue", "Da Nang"][i % 3],
      status: ["paid", "pending"][i % 2],
      amount: Math.floor(Math.random() * 100)
    })
  }
< {
    acknowledged: true,
    insertedId: ObjectId('6ae4cc2df057d3ff0424a513')
  }
shop >
```

- Explain trước khi có index:

```
> db.orders_big.find({ city: "Hanoi", status: "paid" }).explain("executionStats")
< {
    explainVersion: '1',
    queryPlanner: {
      namespace: 'shop.orders_big',
      parsedQuery: {
        '$and': [
          {
            city: {
              '$eq': 'Hanoi'
            }
          },
          {
            status: {
              '$eq': 'paid'
            }
          }
        ]
      },
      indexFilterSet: false,
      queryHash: '10A92AE3',
      planCacheShapeHash: '10A92AE3',
      planCacheKey: 'D5F2F1A1',
      optimizationTimeMillis: 2,
      maxIndexedOrSolutionsReached: false,
```

{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'shop.orders_big',
    parsedQuery: {
      '$and': [
        {
          city: {
            '$eq': 'Hanoi'

```
        }
       },
       {
        status: {
         '$eq': 'paid'
        }
       }
      ]
     },
     indexFilterSet: false,
     queryHash: '10A92AE3',
     planCacheShapeHash: '10A92AE3',
     planCacheKey: 'D5F2F1A1',
     optimizationTimeMillis: 2,
     maxIndexedOrSolutionsReached: false,
     maxIndexedAndSolutionsReached: false,
     maxScansToExplodeReached: false,
     prunedSimilarIndexes: false,
     winningPlan: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
       '$and': [
        {
         city: {
          '$eq': 'Hanoi'
         }
        },
        {
         status: {
          '$eq': 'paid'
         }
        }
       ]
      },
      nss: 'shop.orders_big',
      direction: 'forward'
     },
     rejectedPlans: []
    },
    executionStats: {
     executionSuccess: true,
     nReturned: 1667,
     executionTimeMillis: 17,
```

```
    totalKeysExamined: 0,
    totalDocsExamined: 10000,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        '$and': [
          {
            city: {
              '$eq': 'Hanoi'
            }
          },
          {
            status: {
              '$eq': 'paid'
            }
          }
        ]
      },
      nReturned: 1667,
      executionTimeMillisEstimate: 14,
      works: 10001,
      advanced: 1667,
      needTime: 8333,
      needYield: 0,
      saveState: 1,
      restoreState: 1,
      isEOF: 1,
      nss: 'shop.orders_big',
      direction: 'forward',
      docsExamined: 10000
    }
  },
  queryShapeHash:
'AFC747BB67C051FFF83D19D2F165A1C9B6C42A5118C54EF33A9A4E278908F5BB',
  command: {
    find: 'orders_big',
    filter: {
      city: 'Hanoi',
      status: 'paid'
    },
    '$db': 'shop'
  },
  serverInfo: {
```

```
  host: '911c956cb899',
  port: 27017,
  version: '8.3.13',
  gitVersion: '3b8a5e1f78cf7b94c40a78ad3fe93708cdca8323'
 },
 serverParameters: {
  internalQueryFacetBufferSizeBytes: 104857600,
  internalDocumentSourceGroupMaxMemoryBytes: 104857600,
  internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
  internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
  internalQueryFacetMaxOutputDocSizeBytes: 104857600,
  internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
  internalQueryProhibitBlockingMergeOnMongoS: 0,
  internalQueryMaxAddToSetBytes: 104857600,
  internalQueryFrameworkControl: 'trySbeRestricted',
  internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
 },
 ok: 1
}
```

- Tạo index:

```
> db.orders_big.createIndex({ city: 1, status: 1 })
< city_1_status_1
shop>
```

- Explain lại sau khi có index:

```
> db.orders_big.find({ city: "Hanoi", status: "paid" }).explain("executionStats")
< {
   explainVersion: '1',
   queryPlanner: {
     namespace: 'shop.orders_big',
     parsedQuery: {
       '$and': [
         {
           city: {
             '$eq': 'Hanoi'
           }
         },
         {
           status: {
             '$eq': 'paid'
           }
         }
       ]
     },
     indexFilterSet: false,
     queryHash: '10A92AE3',
     planCacheShapeHash: '10A92AE3',
     planCacheKey: '7D3B8921',
     optimizationTimeMillis: 0,
     maxIndexedOrSolutionsReached: false,
```

```
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'shop.orders_big',
    parsedQuery: {
      '$and': [
        {
          city: {
            '$eq': 'Hanoi'
          }
        },
        {
          status: {
            '$eq': 'paid'
          }
        }
      ]
    },
    indexFilterSet: false,
    queryHash: '10A92AE3',
    planCacheShapeHash: '10A92AE3',
    planCacheKey: '7D3B8921',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'FETCH',
      nss: 'shop.orders_big',
      inputStage: {
        stage: 'IXSCAN',
        nss: 'shop.orders_big',
        keyPattern: {
          city: 1,
          status: 1
        },
        indexName: 'city_1_status_1',
        isMultiKey: false,
        multiKeyPaths: {
          city: [],
          status: []
        },
```

```
      isUnique: false,
      isSparse: false,
      isPartial: false,
      indexVersion: 2,
      direction: 'forward',
      indexBounds: {
        city: [
          '["Hanoi", "Hanoi"]'
        ],
        status: [
          '["paid", "paid"]'
        ]
      }
    }
  },
  rejectedPlans: []
},
executionStats: {
  executionSuccess: true,
  nReturned: 1667,
  executionTimeMillis: 16,
  totalKeysExamined: 1667,
  totalDocsExamined: 1667,
  executionStages: {
    isCached: false,
    stage: 'FETCH',
    nReturned: 1667,
    executionTimeMillisEstimate: 10,
    works: 1668,
    advanced: 1667,
    needTime: 0,
    needYield: 0,
    saveState: 0,
    restoreState: 0,
    isEOF: 1,
    nss: 'shop.orders_big',
    docsExamined: 1667,
    alreadyHasObj: 0,
    inputStage: {
      stage: 'IXSCAN',
      nReturned: 1667,
      executionTimeMillisEstimate: 10,
      works: 1668,
      advanced: 1667,
```

```
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        nss: 'shop.orders_big',
        keyPattern: {
          city: 1,
          status: 1
        },
        indexName: 'city_1_status_1',
        isMultiKey: false,
        multiKeyPaths: {
          city: [],
          status: []
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          city: [
            '["Hanoi", "Hanoi"]'
          ],
          status: [
            '["paid", "paid"]'
          ]
        },
        keysExamined: 1667,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0,
        peakTrackedMemBytes: 0
      }
    }
  },
  queryShapeHash:
'AFC747BB67C051FFF83D19D2F165A1C9B6C42A5118C54EF33A9A4E278908F5BB',
  command: {
    find: 'orders_big',
    filter: {
      city: 'Hanoi',
      status: 'paid'
```

```
    },
    '$db': 'shop'
  },
  serverInfo: {
    host: '911c956cb899',
    port: 27017,
    version: '8.3.13',
    gitVersion: '3b8a5e1f78cf7b94c40a78ad3fe93708cdca8323'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 104857600,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1
}
```


Trước khi tạo index, truy vấn {city: "Hanoi", status: "paid"} dùng COLLSCAN, phải quét toàn bộ totalDocsExamined: 10000 document để trả về 1667 kết quả (17 ms). Sau khi tạo index {city: 1, status: 1}, truy vấn dùng IXSCAN và chỉ đọc totalDocsExamined: 1667 document, đúng bằng số document khớp điều kiện (16 ms). Index giúp giảm số document phải kiểm tra từ 10000 xuống 1667. Do dữ liệu nhỏ nên chênh lệch thời gian không đáng kể, nhưng lợi ích sẽ rõ hơn khi collection lớn.